FINAL REPORT

Tunisia Real Estate AI

Final Technical Report

End-to-end machine learning pipeline, geospatial pricing system, interactive atlas, and AI-assisted technical workflow for Tunisia real estate intelligence

Project Scope

This report documents the complete `RealEstate_V2` system from raw dataset discovery to production-facing APIs, nationwide delegation coverage, model training, AI narration, notebook copilot assistance, and deployment-ready frontend exports.

Project	Tunisia Real Estate Price Prediction System
Repository structure	Pipeline + FastAPI + Interactive Atlas + Notebook UI
Primary task	Predict property prices with full delegation-level geographic reach
Best model	Gradient Boosting Regressor
Final feature count	10 model features in 3 groups
Coverage objective	264 / 264 delegations across Tunisia
Deployment style	Production API with frontend and AI-enhanced notebook experience

Realised by

Mohamed Aziz Mansour • Rimel Zouari • Mohamed Ncib

Prepared from the implemented codebase, generated reports, notebook chapters, and deployed interface artifacts

April 14, 2026

Contents

1 Executive Summary **3**

2 Problem Statement and Objectives **3**

2.1 Why the Problem Matters . . . 3

2.2 Project Objectives . . . 3

3 System Architecture **4**

3.1 Primary Components . . . 4

4 Data Sources and Geographic Reference **4**

4.1 Raw Property Sources . . . 4

4.2 Discovery Snapshot of the Raw Sources . . . 5

4.3 Geographic Reference Assets . . . 5

5 Pipeline Methodology **5**

5.1 Stage 1: Dataset Discovery . . . 5

5.2 Stage 2: Dataset Cleaning . . . 6

5.2.1 Cleaning Results and Merge Readiness . . . 6

5.2.2 What Cleaning Removed in Practice . . . 6

5.3 Stage 3: Merge Preparation . . . 6

5.3.1 Why Merge Preparation Is Its Own Stage . . . 7

5.4 Stage 4: Geographic Name Alignment . . . 7

5.4.1 Examples of Geographic Difficulty . . . 8

5.5 Stage 5: Training Dataset Preparation . . . 8

5.5.1 Why Log Transformations Matter . . . 9

5.6 Stage 6: Feature Engineering . . . 9

5.6.1 Stage 6 Design Principle . . . 10

5.7 Stage 7: Visual Check . . . 10

5.7.1 Why This Stage Matters . . . 10

5.8 Stage 8: Model Training . . . 10

5.8.1 Why Gradient Boosting Won . . . 11

6 Fallback Coverage Strategy **11**

6.1 Operational Interpretation of Coverage . . . 12

7 Model Performance and Interpretation **12**

7.1 Core Metrics . . . 12
7.2 What the Metrics Mean . . . 12
7.3 Feature Design Logic . . . 13
7.4 Coordinate and Geographic Signal Quality . . . 13

8 Frontend and User Experience Layer **13**
8.1 Geographic Atlas . . . 13
8.2 Notebook Companion . . . 14
8.3 Notebook Chapter Structure . . . 14

9 AI Features Integrated into the Project **15**
9.1 AI Narrator . . . 15
9.2 Notebook Cell Copilot . . . 15

10 API Layer **16**
10.1 Notable API Responsibilities . . . 16

11 Deployment-Ready Artifacts **16**
11.1 Processed Data Artifacts . . . 16
11.2 Model and Frontend Artifacts . . . 17
11.3 Why Artifact Discipline Matters . . . 17

12 Visual Diagnostics **17**

13 Engineering Improvements Completed During Finalization **17**
13.1 Representative Fixes . . . 18

14 Strengths of the Final System **18**

15 Limitations and Future Work **18**

16 Conclusion **19**

A Key Final Metrics Snapshot **20**

B Project File Highlights **20**

C Suggested Compilation Workflow **20**

D Compilation Note **21**

1 Executive Summary

This project builds a full machine learning system for Tunisian real estate pricing. The final platform does not stop at model training: it integrates structured data engineering, geographic normalization, model evaluation, fallback coverage logic, deployment APIs, a visual atlas, a chapter-based notebook interface, and Groq-powered AI assistance.

The core business problem is geographic heterogeneity. A property of the same size can vary significantly in value between Tunis, Ariana, Nabeul, Sousse, Sfax, or lower-density interior governorates. A direct regression model trained only on high-density delegations would leave large parts of the country unsupported. The implemented system addresses this through a hybrid strategy: direct model support where training data is strong, and hierarchical fallback benchmarks where local evidence is limited.

The final production model is a **Gradient Boosting Regressor** trained on **13,651 modeling rows** after final outlier filtering. It reaches a **test R^2 of 0.8868**, a **cross-validation R^2 of 0.9279 $\pm$ 0.0039**, and a **test MAE of 70,949.98 TND**. Most importantly, the prediction and atlas logic reach **100% national delegation coverage** across **264 delegations**.

The system is also notable for its presentation layer. The frontend atlas exposes direct versus fallback support, model metrics, benchmark-driven scenario pricing, and a Groq-based AI narrator. The notebook companion provides runnable cells, project diagnostics, and a per-cell Groq copilot that can explain code, outputs, and errors with full contextual grounding.

2 Problem Statement and Objectives

2.1 Why the Problem Matters

Real estate valuation is deeply spatial. In Tunisia, prices depend not only on intrinsic variables such as surface area and room count, but also on governorate, delegation, neighborhood structure, market depth, and the amount of historical evidence available in a given area. This creates several practical challenges:

- the same property profile can have radically different prices depending on location,
- raw listing data arrives from heterogeneous sources with inconsistent schema and quality,
- locality names are noisy, multilingual, and often misspelled,
- many zones do not have strong direct training support,
- a user-facing system must still offer coherent estimates across all delegations.

2.2 Project Objectives

The project therefore had five concrete objectives:

1. build a reliable multi-stage data pipeline from raw property listings to model-ready data,
2. normalize location names against official Tunisia geography,
3. train a strong regression model while preserving interpretability of coverage and support,
4. provide complete atlas-level coverage through fallback benchmarks,

5. expose the result through a production-style API and polished frontend interfaces.

3 System Architecture

The implemented solution is organized around three tightly connected layers:

Layer	Description
Data Pipeline	Eight sequential stages transform raw listings into cleaned, merged, geo-aligned, feature-engineered, and model-ready artifacts.
Prediction API	FastAPI serves prediction requests, model summaries, delegation coverage, notebook execution, source rendering, and Groq-backed AI endpoints.
Geographic Atlas and Notebook UI	The frontend includes an atlas for market exploration and a notebook companion for chapter-by-chapter technical walkthrough.

The architecture is artifact-centered rather than purely code-centered. Each pipeline stage writes files under `data/processed/`, and those files are then reused by the model, the API, the atlas, and the notebook dashboard. This design keeps the project consistent across training and serving.

3.1 Primary Components

- `pipeline/`: numbered Python stages from discovery to model training,
- `data/processed/`: reports and generated datasets per stage,
- `artifacts/08_best_model.joblib`: deployed trained model,
- `api/app.py`: serving layer and AI endpoints,
- `frontend/index.html`: atlas interface,
- `frontend/notebook.html`: workflow notebook UI,
- `frontend/model_summary.json`: frontend metrics snapshot.

4 Data Sources and Geographic Reference

4.1 Raw Property Sources

The system uses three listing datasets located in `data/raw/`:

- `tunisia-real-estate.csv`: the largest source and the only one with strong land coverage,
- `Property Prices in Tunisia.csv`: a secondary source with strong apartment and house signal,
- `data_prices_cleaned.csv`: a pre-cleaned subset with useful high-density zone coverage.

These datasets use different schemas, naming conventions, and quality profiles. The pipeline standardizes them into a unified representation before merging.

4.2 Discovery Snapshot of the Raw Sources

The three input files are not equivalent. Stage 1 reveals major differences in row count, schema width, duplicates, and missingness.

Dataset	Rows	Columns	Exact duplicates	Price missing	Surface missing
tunisia_real_estate	25,317	8	0	0	0
property_prices_in_tunisia	12,748	9	1,613	0	0
data_prices_cleaned	8,336	17	0	14	1,584

Additional discovery observations are important for understanding later cleaning choices:

- `tunisia_real_estate` has the largest raw footprint and includes strong land representation, but some room and bathroom values are naturally sparse for land listings.
- `property_prices_in_tunisia` contains a significant duplicate burden and many non-positive surface values at discovery time, indicating the need for aggressive filtering.
- `data_prices_cleaned` is structurally richer, but it contains meaningful missingness in price and especially surface area, which makes quality control essential before merge.

The discovery stage therefore does more than summarize data. It tells the engineering team where reliability is high, where cleaning effort must be concentrated, and which source-specific assumptions are safe.

4.3 Geographic Reference Assets

The project also relies on official or semi-official geographic reference files under `geo/`, including delegation boundaries and administrative naming resources. These files are essential for:

- canonical governorate and delegation naming,
- centroid extraction for coordinates,
- atlas rendering,
- delegation-level support and fallback coverage exports.

5 Pipeline Methodology

The complete workflow is implemented as eight sequential stages. The notebook companion mirrors the same stages with explicit chapter boundaries and runnable cells.

5.1 Stage 1: Dataset Discovery

Stage 1 profiles the three raw datasets to understand schema, counts, missingness, and detection of core fields such as price, surface, location, and property type. This stage establishes the data quality baseline and reveals heterogeneity across sources.

The discovery stage matters because every downstream assumption depends on what was observed here. If one dataset stores price as text with currency suffixes while another stores it

numerically, that difference has to be normalized before any meaningful merge or model training can happen.

5.2 Stage 2: Dataset Cleaning

Stage 2 standardizes text, strips noise, repairs encoding corruption, coerces numeric fields, filters obviously invalid rows, and prepares the datasets for merge-readiness. This is the stage where the project converts raw listings into structured rows that can participate in a common pipeline.

The cleaning logic is especially important because this project uses mixed sources. Without aggressive normalization, the downstream merge and alignment stages would amplify noise rather than reduce it.

5.2.1 Cleaning Results and Merge Readiness

By the end of Stage 2, the datasets are reduced to merge-ready subsets with all critical issue counts driven to zero.

Dataset	Clean rows	Duplicate rows	Invalid family rows	Missing core fields
tunisia_real_estate	9,166	0	0	0
property_prices_in_tunisia	2,920	0	0	0
data_prices_cleaned	1,745	0	0	0

This is a strong indicator of pipeline quality. It means Stage 3 can assume a stable unified schema instead of repeatedly defending itself against missing or malformed rows.

5.2.2 What Cleaning Removed in Practice

The drop from raw to clean rows is not accidental. It reflects a deliberate set of quality rules:

- removal of duplicate listings,
- elimination of invalid surface values,
- normalization of property family labels,
- filtering of rows missing required fields for later geospatial and modeling steps,
- text normalization across governorate, delegation, city, and location fields.

This decision is important from a modeling perspective. A smaller but trustworthy dataset is far more useful than a larger noisy dataset when target encodings, fallback support, and geographic benchmarks are all downstream consumers of the same records.

5.3 Stage 3: Merge Preparation

Stage 3 unifies the cleaned datasets into one training corpus with consistent columns and source priority. The generated merge report shows:

- input rows: 13,722,

- rows removed as cross-source duplicates: 3,
- final merged rows: 13,719.

The source distribution after merge is shown below.

Source	Rows	Share
tunisia_real_estate	9,038	65.88%
property_prices_in_tunisia	2,920	21.29%
data_prices_cleaned	1,761	12.83%

The small number of cross-source duplicates is a good signal: it means the merge stage preserved most rows while still performing useful de-duplication.

5.3.1 Why Merge Preparation Is Its Own Stage

Separating merge preparation from cleaning is a good architectural decision. Cleaning solves source-local issues, while merge preparation solves cross-source issues. This distinction improves maintainability because:

- source-specific transformations remain localized,
- merge-time logic can focus on schema unification and source priority,
- duplicate handling across sources can be inspected independently,
- stage reports remain interpretable instead of mixing unrelated concerns.

5.4 Stage 4: Geographic Name Alignment

Stage 4 maps listing geography to canonical Tunisia governorate and delegation names. This stage is one of the most important in the entire pipeline because geography is the strongest signal in the final model.

The alignment report gives the following breakdown:

- canonical delegations in reference geography: 264,
- dataset rows entering alignment: 13,788,
- matched rows: 13,756,
- unmatched rows: 32,
- unique covered delegations in the dataset: 181.

Matching happened through several strategies:

Match status	Rows
exact_delegation	7,947
manual_override	4,684
exact_locality	995
fuzzy_delegation	92
substring_locality	38
unmatched	32

This stage demonstrates a practical engineering mindset: the project does not rely on one matching strategy. Instead, it layers exact matching, fuzzy matching, locality clues, and manual overrides to maximize geographic resolution while preserving correctness.

5.4.1 Examples of Geographic Difficulty

The unmatched list shows why this stage is non-trivial. Remaining unresolved cases include values like `Ezzouhour (tunis)`, `Le Kef Ouest`, `Jendouba`, `Essijoumi`, and `Bir El Hafey`. These examples reflect a mix of alternate administrative naming, locality/delegation ambiguity, and spelling variants.

In practice, this confirms that geographic alignment is not merely string cleaning. It is a semantic normalization problem that requires domain-aware aliasing and structured fallback logic.

5.5 Stage 5: Training Dataset Preparation

Stage 5 converts the aligned dataset into the training-ready table used for model construction. It filters rows with valid core fields and creates derived price fields such as:

- `price_per_m2`,
- `log_price_tnd`,
- `log_price_per_m2`.

The resulting training dataset contains:

- rows: 13,788,
- governorates: 24,
- delegations: 181,
- normalized localities: 293.

Property family distribution in the final Stage 5 dataset:

Property family	Rows
apartment	5,459
house	4,944
land	3,385

Observed price range:

- minimum: 8,400 TND,
- median: 318,000 TND,
- maximum: 15,600,000 TND.

5.5.1 Why Log Transformations Matter

The project computes both `log_price_tnd` and `log_price_per_m2`. This is an important modeling choice because real estate prices usually exhibit a long-tailed distribution. A small number of expensive listings can dominate the scale if the target is left in raw form.

Using log-space has two concrete benefits:

- it reduces the skew of the target distribution,
- it helps tree-based models learn smoother price relationships across very different market segments.

The notebook visualizations confirm this logic by explicitly comparing raw price distribution against log-price distribution.

5.6 Stage 6: Feature Engineering

Stage 6 writes the base engineered dataset and attaches canonical geographic coordinates. After the recent synchronization fixes, Stage 6 is intentionally limited to the features that truly belong to this stage:

- `surface_m2`,
- `rooms`,
- `lon`,
- `lat`.

The project originally had a coordinate coverage issue caused by mismatch between canonical delegation names and raw GeoJSON naming. That problem was fixed by aligning coordinate generation with canonical delegation keys. The final Stage 6 report confirms:

- rows with geo delegation: 13,756,
- rows with coordinates: 13,756,
- rows missing coordinates despite geo match: 0,
- remaining missing coordinates: 32, corresponding to truly unmatched geography rows.

This fix is important because the atlas and model both depend on accurate geographic coordinate attribution.

5.6.1 Stage 6 Design Principle

The final version of the project deliberately keeps Stage 6 narrow in scope. It writes only the base engineered dataset and does not pretend to include split-dependent features that only become valid during model training.

This separation is a strong engineering improvement because it avoids a common reporting mistake: claiming that the feature-engineered dataset already contains target encodings when in reality those encodings must be created after the train/test split.

5.7 Stage 7: Visual Check

Stage 7 performs a diagnostic quality pass on the training dataset. It summarizes each source contribution after the structured pipeline has already standardized and aligned the data.

Examples from the visual check report:

- `tunisia_real_estate`: 9,166 rows, 24 governorates, 159 delegations,
- `property_prices_in_tunisia`: 2,906 rows, 23 governorates, 145 delegations,
- `data_prices_cleaned`: 1,716 rows, 18 governorates, 91 delegations.

This stage is not model training. It is a quality assurance checkpoint designed to catch imbalances or suspicious source behavior before fitting the final estimator.

5.7.1 Why This Stage Matters

Many pipelines skip this kind of intermediate validation and move directly from feature engineering to model training. This project does not. That is valuable because visual QA provides a human-readable sanity layer between technical transformation and statistical fitting. It helps answer practical questions such as:

- Which source dominates a given delegation?
- Are median prices consistent with expectations?
- Is a given source over-concentrated in a small number of coastal zones?
- Are property family distributions plausible after cleaning and merge?

5.8 Stage 8: Model Training

Stage 8 introduces the final split-dependent features and trains the actual regressors. The final feature groups are:

- base numeric: `surface_m2`, `rooms`, `lon`, `lat`,
- split-dependent numeric: `geo_delegation_target_enc`, `geo_governorate_target_enc`, `price_vs_local_median`,
- categorical: `property_family`, `geo_governorate`, `geo_delegation`.

The project compares at least two models:

Model	Test R^2	Test MAE (TND)
GradientBoosting	0.8868	70,949.98
RandomForest	0.8560	76,495.28

The winning model is therefore **GradientBoosting**. This final stage also writes the model artifact, holdout predictions, plots, coverage exports, and frontend summary assets.

5.8.1 Why Gradient Boosting Won

The final comparison suggests that Gradient Boosting captures the relationship between structural and geographic features more effectively than the Random Forest baseline in this project. A likely reason is that the final feature space includes a mixture of:

- smooth numeric variables,
- split-dependent encoded geographic priors,
- categorical identity features,
- fallback-sensitive geographic context.

Gradient Boosting often performs particularly well in this kind of structured tabular setting where a relatively compact feature set contains strong, non-linear interactions.

6 Fallback Coverage Strategy

One of the strongest design choices in the project is the separation between **model quality** and **national usability**. Even with a strong model, some delegations do not have enough direct support to justify a purely local estimate. The project addresses this using hierarchical fallback tiers.

The final production summary reports:

- total delegations: 264,
- covered delegations: 264,
- delegations with direct support: 103,
- direct coverage: 39.02%,
- fallback coverage: 60.98%,
- atlas reach: 100%.

Tier percentages:

Coverage tier	Share
exact_sector	19.70%
governorate_fallback	61.74%
national_fallback	18.56%

This architecture gives the system a practical advantage over many purely local regressors: it can still produce a coherent market benchmark everywhere, while transparently distinguishing zones with direct evidence from zones using broader fallback support.

6.1 Operational Interpretation of Coverage

Coverage is not just a reporting metric. It affects how the atlas should be interpreted by end users:

- direct support means the selected area is backed by strong local evidence,
- fallback support means the estimate is still usable but is more benchmark-driven,
- national fallback should be interpreted as a broad market proxy rather than a precise local estimate.

This explicit separation improves trust because the system does not present all predictions with equal implied certainty.

7 Model Performance and Interpretation

7.1 Core Metrics

Final production metrics from the current artifacts are:

- training rows: 13,788,
- modeling rows after final trim: 13,651,
- test R^2: 0.8868176804,
- validation R^2: 0.8868176804,
- 5-fold CV mean R^2: 0.9278765811,
- 5-fold CV standard deviation: 0.0039254350,
- test MAE: 70,949.98 TND.

These numbers suggest a model that is both strong and stable. The cross-validation variance is low, which indicates that the learned structure generalizes well across training folds.

7.2 What the Metrics Mean

The test R^2 of roughly 0.887 means the model explains a large proportion of variance in property prices on holdout data. In a noisy real estate environment with heterogeneous source quality, this is a strong result.

The MAE of approximately 71k TND must be interpreted relative to the data range. Since prices span from 8.4k TND to 15.6M TND, the absolute error is not negligible but remains reasonable for a national-scale pricing system with fallback coverage and mixed source quality.

7.3 Feature Design Logic

The final feature set is carefully engineered to combine structural and geographic signals.

Feature group	Role
Base numeric features	Capture property size and geographic centroid position.
Split-dependent numeric features	Capture local price priors without leaking test information into training.
Categorial geographic features	Preserve delegation and governorate identity as explicit model inputs.

The use of split-dependent target encodings is especially important. The project correctly distinguishes Stage 6 base features from Stage 8 train-split-safe features, preventing leakage while preserving strong local pricing signal.

7.4 Coordinate and Geographic Signal Quality

The final model report confirms that geographic coordinates are now fully synchronized with the matched geography rows before imputation:

- rows with geo delegation before imputation: 13,756,
- rows with coordinates before imputation: 13,756,
- rows missing coordinates despite geo match before imputation: 0.

This matters because poor coordinate attribution would silently weaken one of the strongest spatial signals in the model.

8 Frontend and User Experience Layer

8.1 Geographic Atlas

The atlas frontend is not just a static map. It is a decision interface that exposes:

- delegation selection,
- support versus fallback map modes,
- family-specific scenario pricing,
- confidence and coverage metadata,
- AI-generated narration for the selected area.

The atlas is backed by generated frontend exports:

- `frontend/assets/data/atlas.geojson`,
- `frontend/assets/data/zone_coverage.json`,
- `frontend/assets/data/delegation_profiles.json`,

- `frontend/model_summary.json`.

This export strategy is excellent for deployment because the API, the UI, and the model report all read from a consistent artifact set.

8.2 Notebook Companion

The notebook UI has evolved into a complete technical companion rather than a static notebook dump. It now provides:

- 13 visible sections aligned with the notebook chapters,
- runnable code cells,
- dashboard diagnostics,
- source viewers,
- per-cell Groq copilot assistance.

This is a major strength of the project. It bridges the gap between a data science notebook and a production-grade workflow viewer.

8.3 Notebook Chapter Structure

The notebook workflow is organized into 13 visible sections:

1. Project Overview,
2. Environment Setup,
3. Utility Functions,
4. Data Sources,
5. Stage 1: Dataset Discovery,
6. Stage 2: Dataset Cleaning,
7. Stage 3: Merge Preparation,
8. Stage 4: Geographic Name Alignment,
9. Stage 5: Training Dataset Preparation,
10. Stage 6: Feature Engineering,
11. Stage 7: Visual Check,
12. Stage 8: Model Training,
13. Conclusion.

That structure is important because it keeps the technical narrative aligned with the actual pipeline boundaries. It also helps learners and reviewers understand not just what the final model is, but how the project arrives there.

9 AI Features Integrated into the Project

9.1 AI Narrator

The atlas includes a Groq-powered AI narrator that generates delegation-aware market insight. Important implementation characteristics include:

- AI-only frontend narration path,
- structured SSE streaming,
- visible provider/model metadata,
- no static frontend fallback templates,
- automatic regeneration when the user changes zone or scenario.

Current narrator model:

- provider: Groq,
- model: `llama-3.3-70b-versatile`.

The streaming logic was improved to use explicit SSE events for stream start, token delivery, completion, and error handling. This makes the frontend narrator more reliable and more transparent than a naive chunk-concatenation implementation.

9.2 Notebook Cell Copilot

The notebook now includes a premium per-cell AI assistant. Each cell can open an inline AI panel that sends:

- chapter title,
- explainer text,
- exact code,
- current output,
- current error state,
- local message history.

This makes the notebook not only inspectable but conversational. It can explain, debug, and interpret pipeline logic at cell granularity, which is highly valuable for learning, demonstrations, and maintenance.

The design quality of this feature comes from grounding. The model is not asked vague questions about the project in general. It is asked about a specific cell with its surrounding context, which greatly improves answer precision.

10 API Layer

The FastAPI backend serves as the operational bridge between model artifacts and user interfaces. It provides endpoints for:

- property price prediction,
- model summary retrieval,
- delegation coverage access,
- notebook source rendering,
- notebook cell execution,
- atlas AI narration,
- notebook cell AI chat.

This unified API design is strategically sound because it keeps all interfaces grounded in the same project files and metrics.

10.1 Notable API Responsibilities

The API is responsible for more than prediction. It also acts as a runtime bridge between offline pipeline artifacts and interactive frontend experiences. In effect, it supports three distinct workloads:

- numerical prediction and coverage lookup,
- notebook execution and source inspection,
- AI-assisted explanation through Groq-backed streams.

This multifunction role makes the API a central orchestration layer rather than a thin model wrapper.

11 Deployment-Ready Artifacts

The project writes a rich set of outputs that support not only training but also deployment and explanation.

11.1 Processed Data Artifacts

- discovery profiles under `data/processed/01_discovery/`,
- cleaned datasets under `data/processed/02_cleaning/`,
- merge report and dataset under `data/processed/03_merge/`,
- alignment outputs under `data/processed/04_geo_alignment/`,
- training dataset outputs under `data/processed/05_training_dataset/`,

- feature engineering outputs under `data/processed/06_feature_engineering/`,
- visual check outputs under `data/processed/07_visual_check/`,
- model reports and figures under `data/processed/08_model_training/`.

11.2 Model and Frontend Artifacts

- trained model: `artifacts/08_best_model.joblib`,
- frontend metrics: `frontend/model_summary.json`,
- atlas support file: `frontend/assets/data/zone_coverage.json`,
- delegation profiles: `frontend/assets/data/delegation_profiles.json`.

This makes the project easy to reason about operationally: there is a clear chain from raw data to processed data to model artifact to deployed UI asset.

11.3 Why Artifact Discipline Matters

The project benefits from writing reports at every stage. This has several practical advantages:

- debugging becomes easier because intermediate failures are inspectable,
- frontend metrics can be grounded in generated JSON rather than duplicated logic,
- notebook explanations can cite real artifacts rather than hand-written summaries,
- final reporting becomes much more reliable because the evidence already exists on disk.

12 Visual Diagnostics

The model training stage exports visual diagnostics such as model comparison and holdout scatter plots. If the files are present at compile time, they are included below.

Model comparison figure not found at compile time.

Holdout scatter figure not found at compile time.

These visual exports strengthen the credibility of the project because they move evaluation beyond a single scalar metric. They show the model comparison process and the shape of holdout behavior.

13 Engineering Improvements Completed During Finalization

The project was not frozen as a simple pipeline run; several quality improvements were completed during final synchronization:

- Stage 6 coordinate assignment was fixed by using canonical delegation keys instead of raw GeoJSON names.

- The feature story was synchronized across pipeline scripts, notebook source, notebook HTML, and generated reports.
- `frontend/notebook.html` was restructured to follow the notebook chapter map more closely.
- The notebook table parser was improved so structured discovery outputs render correctly in the frontend.
- The AI narrator was upgraded to true SSE event streaming with explicit model metadata.
- A per-cell Groq notebook copilot was added with inline streamed explanations.

These changes matter because they raise the project from a strong ML pipeline to a polished product-quality experience.

13.1 Representative Fixes

The finalization phase is worth documenting because it reflects real engineering practice. Some key improvements include:

- synchronizing the notebook source, notebook HTML, and pipeline feature reports,
- correcting chapter boundaries in `notebook.html` so they match the true notebook stages,
- improving the notebook renderer so printed structured outputs are parsed correctly,
- making the AI narrator fully model-driven in the frontend path,
- exposing the actual Groq model metadata directly in the interface.

14 Strengths of the Final System

The project has several strong qualities:

1. **End-to-end completeness**. The system includes pipeline, model, API, atlas, notebook, and AI assistance.
2. **Artifact consistency**. Frontend and backend both consume generated reports rather than disconnected hand-written summaries.
3. **Geographic realism**. Delegation support and fallback logic are explicit and operational.
4. **High usability**. Users can inspect both model outputs and pipeline internals.
5. **Good engineering recovery**. Important issues such as coordinate mismatch and notebook drift were identified and corrected.

15 Limitations and Future Work

No production system is perfect, and there are still useful next steps:

- improve support depth in low-data delegations,

- enrich locality-level market features beyond current fallback tiers,
- add explicit uncertainty estimation rather than confidence labels tied mainly to coverage support,
- extend model explainability with prediction-level AI rationales,
- add pairwise delegation comparison and dashboard AI analyst capabilities,
- package the report and notebook pipeline into reproducible CI checks.

An especially strong next feature would be a Groq-powered *Why this prediction?* explainer on the atlas, grounded in the exact benchmark, coverage tier, selected family, and scenario surface.

Other strong future extensions include:

- pairwise delegation comparison using AI-generated market contrast narratives,
- chapter-level notebook summaries generated from stage reports,
- richer uncertainty quantification for benchmark fallback predictions,
- CI-based regression checks on generated reports and frontend exports,
- packaged PDF report generation from the final artifacts.

16 Conclusion

This project successfully delivers a complete Tunisia real estate pricing system with national delegation coverage, strong model accuracy, structured geospatial alignment, deployment-ready artifacts, and polished user-facing interfaces.

From a technical perspective, the project demonstrates more than regression modeling. It shows practical ML engineering: data cleaning, schema unification, geospatial normalization, coordinate repair, split-safe feature engineering, artifact export design, API serving, and UI-grounded AI assistance.

From a product perspective, the final result is compelling because it combines:

- a strong prediction engine,
- transparent geographic support logic,
- interactive atlas exploration,
- explainable notebook workflow,
- premium Groq-powered AI assistance.

In short, the final system is not just a trained model. It is a coherent machine learning product for Tunisia real estate intelligence.

A Key Final Metrics Snapshot

Metric	Value
Best model	GradientBoosting
Training rows	13,788
Modeling rows	13,651
Test R^2	0.8868
Cross-validation R^2 mean	0.9279
Cross-validation R^2 std	0.0039
Test MAE	70,949.98 TND
Total delegations	264
Covered delegations	264
Delegations with direct support	103
Direct coverage	39.02%
Fallback coverage	60.98%
Atlas reach	100%
Final feature count	10
AI narrator model	Groq / `llama-3.3-70b-versatile`
Notebook copilot model	Groq / `llama-3.3-70b-versatile`

B Project File Highlights

- `pipeline/01_dataset_discovery.py`
- `pipeline/02_dataset_cleaning.py`
- `pipeline/03_merge_preparation.py`
- `pipeline/04_geo_name_alignment.py`
- `pipeline/05_training_dataset_preparation.py`
- `pipeline/06_feature_engineering.py`
- `pipeline/07_training_dataset_visual_check.py`
- `pipeline/08_model_training.py`
- `api/app.py`
- `frontend/index.html`
- `frontend/notebook.html`
- `frontend/js/map.js`
- `frontend/js/notebook.js`

C Suggested Compilation Workflow

For a final deliverable, the recommended workflow is:

1. compile `rapport.tex` in Overleaf or a local TeX distribution,

2. verify that the generated plots are found under `data/processed/08_model_training/`,
3. export the PDF alongside the code repository as the final written report,
4. optionally add institutional branding or supervisor metadata on the title page.

D Compilation Note

This report is designed as a single-file LaTeX document named `rapport.tex`. It can be compiled with a standard PDF LaTeX toolchain or on Overleaf. Figures are included conditionally, so missing image files will not break compilation.